⬡ Meridian

ROLE GUIDE & IMPLEMENTATION SPEC

P2 · DATA & MEMORY

P2 — Data / Memory Lead

Role: Generate all sample data, build the vector store (ChromaDB), and implement episodic memory (SQLite).

Timeline: 3 days. Work in `backend/data/`, `backend/memory/`, `scripts/`, and `tests/`.

Files You Own

```
backend/
├── data/
│   ├── meeting_notes/          (10 .md files)
│   ├── playbooks/              (15 .md files)
│   ├── customer_profiles/      (5 .json files)
│   └── resolved_cases/         (6 .json files)
├── memory/
│   ├── vector_store.py
│   └── episodic.py
scripts/
├── ingest.py
└── seed_memory.py
tests/
└── eval_scenarios.py
```

Dependencies: `chromadb`, `sentence-transformers`, `sqlite3`, `python-dotenv`, `os`, `json`, `glob`, `uuid`

Interface Contracts

You Provide (to P1)

- `backend/memory/vector_store.py` — `search(query, collection, n=5)`
- `backend/memory/episodic.py` — `get_memory_context(risk_level, risk_score)`

You Provide (to P3)

- `backend/data/customer_profiles/*.json` — loaded by FastAPI
- `scripts/ingest.py` — must be runnable to populate ChromaDB
- `scripts/seed_memory.py` — must be runnable to pre-load SQLite

Day 1 — All Content + ChromaDB Ingestion

1. Meeting transcripts (`data/meeting_notes/`)

Create 10 `.md` files with real dialogue format (speaker labels, specific numbers, product names).

File	Account	Scenario
`acme_corp.md`	Acme Corp	At-risk: VP angry, 47 days to renewal, no reporting module adoption, explicit churn threat
`bluewave_tech.md`	BlueWave Tech	At-risk: CFO questioning ROI, competitor evaluation mentioned by name
`nexus_systems.md`	Nexus Systems	At-risk: key champion left, support tickets spiking 3×
`globex_corp.md`	Globex Corp	Healthy/expansion: power users, asking about enterprise tier, wants API access
`umbrella_corp.md`	Umbrella Corp	Healthy/expansion: great NPS, wants to expand to 3 new teams
`initech.md`	Initech	Healthy/expansion: successful QBR, requesting custom integration scoping
`techcorp.md`	TechCorp	Onboarding: 2 weeks in, struggling with setup but engaged
`quantum_labs.md`	Quantum Labs	Onboarding: on track, early adoption strong
`globex_q1.md`	GlobexQ1	Resolved: was at-risk, EBR worked, renewed
`initech_q4.md`	InitechQ4	Resolved: was at-risk, re-onboarding worked

2. Knowledge articles (`data/playbooks/`)

Create 15 `.md` files:

- `churn_prevention.md`
- `ebr_guide.md`
- `onboarding_best_practices.md`
- `expansion_playbook.md`
- `escalation_procedures.md`
- `health_score_methodology.md`
- `qbr_template.md`
- `champion_change.md`
- `renewal_risk_signals.md`
- `product_reporting_module.md`
- `product_integrations.md`
- `product_automation.md`
- `competitive_positioning.md`
- `success_plan_template.md`
- `sla_policy.md`

Each article must be 300–800 words with realistic CS content.

3. Customer profiles (`data/customer_profiles/`)

Create 5 `.json` files. Filename = account ID (e.g., `acme_corp.json`).

```
{
  "account_name": "Acme Corp",
  "arr": 85000,
  "industry": "Manufacturing",
  "contract_end": "2026-08-15",
  "csm": "Sarah Chen",
  "health_score_history": [72, 68, 61, 55, 48],
  "daily_active_users": 34,
  "last_login_date": "2026-06-20",
  "features_adopted": ["dashboard", "alerts"],
  "open_tickets": 7,
  "nps_score": 22
}
```

Use realistic numbers: ARR $45K–$180K, NPS 20–72, 2–8 open tickets, 5 health scores.

Accounts: `acme_corp`, `bluewave_tech`, `nexus_systems`, `globex_corp`, `techcorp`.

4. `backend/memory/vector_store.py`

```
import chromadb
from sentence_transformers import SentenceTransformer

class VectorStore:
    def __init__(self, persist_dir: str = "./chroma_db"):
        self.client = chromadb.PersistentClient(path=persist_dir)
        self.model = SentenceTransformer("all-MiniLM-L6-v2")
        self._get_or_create("knowledge_base")
        self._get_or_create("resolved_cases")

    def _get_or_create(self, name: str):
        try:
            return self.client.get_collection(name)
        except Exception:
            return self.client.create_collection(name, metadata={"hnsw:space": "cosine"})

    def add(self, texts: list[str], metadatas: list[dict], collection: str):
        coll = self._get_or_create(collection)
        embeddings = self.model.encode(texts).tolist()
        ids = [f"{collection}_{i}" for i in range(len(texts))]
        coll.add(ids=ids, documents=texts, metadatas=metadatas, embeddings=embeddings)

    def search(self, query: str, collection: str, n: int = 5) -> list[dict]:
        coll = self._get_or_create(collection)
        embedding = self.model.encode([query]).tolist()
        results = coll.query(query_embeddings=embedding, n_results=n)
        output = []
        for i in range(len(results["ids"][0])):
            output.append({
                "id": results["ids"][0][i],
                "excerpt": results["documents"][0][i],
                "metadata": results["metadatas"][0][i],
                "distance": results["distances"][0][i],
            })
        return output
```

5. `scripts/ingest.py`

```
#!/usr/bin/env python3
import os, glob
from backend.memory.vector_store import VectorStore

def chunk_text(text: str, max_tokens: int = 500) -> list[str]:
    paragraphs = [p.strip() for p in text.split("

") if p.strip()]
    chunks = []
    for p in paragraphs:
        if len(p.split()) > max_tokens:
            sentences = p.split(". ")
            current = ""
            for s in sentences:
                if len((current + s).split()) > max_tokens:
                    chunks.append(current.strip())
                    current = s
                else:
                    current += ". " + s if current else s
            if current:
                chunks.append(current.strip())
        else:
            chunks.append(p)
    return chunks

def main():
    store = VectorStore()
    for path in glob.glob("backend/data/playbooks/*.md"):
        with open(path) as f:
            text = f.read()
        chunks = chunk_text(text)
        metadatas = [{"source_file": os.path.basename(path), "chunk_index": i, "category": "pla
        store.add(chunks, metadatas, "knowledge_base")
    for path in glob.glob("backend/data/meeting_notes/*.md"):
        with open(path) as f:
            text = f.read()
        chunks = chunk_text(text)
        metadatas = [{"source_file": os.path.basename(path), "chunk_index": i, "category": "mee
        store.add(chunks, metadatas, "knowledge_base")
    count = store._get_or_create("knowledge_base").count()
    print(f"knowledge_base count: {count}")
```

```
    assert count >= 80

if __name__ == "__main__":
    main()
```

Run it and confirm count >= 80.

Day 2 — Resolved Cases + Episodic Memory

6. Resolved cases (`data/resolved_cases/`)

Create 6 `.json` files:

```
{
  "case_id": "CASE-001",
  "account_name": "Globex Corp",
  "arr": 120000,
  "situation_summary": "Champion left, renewal at risk 60 days out.",
  "risk_signals": ["champion departure", "no exec sponsor", "silence > 10 days"],
  "action_taken": "Executive business review with new CTO",
  "outcome": "Renewed with 15% expansion",
  "days_to_resolution": 14,
  "renewal_achieved": true,
  "csm_notes": "EBR was the turning point. New champion emerged."
}
```

Distribution: 4 EBR-resolved, 1 re-onboarding resolved, 1 champion change resolved.

7. `backend/memory/episodic.py`

```
import sqlite3, os
from backend.models.schemas import MemoryContext

DB_PATH = os.getenv("EPISODIC_DB_PATH", "./episodic_memory.db")

class EpisodicMemory:
    def __init__(self):
```

```
        self.conn = sqlite3.connect(DB_PATH, check_same_thread=False)
        self._init_table()

    def _init_table(self):
        self.conn.execute("""
            CREATE TABLE IF NOT EXISTS memory_log (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                account TEXT,
                risk_score REAL,
                risk_level TEXT,
                recommendation_title TEXT,
                decision TEXT,
                modification_notes TEXT,
                outcome TEXT,
                timestamp TEXT
            )
        """)
        self.conn.commit()

    def log_feedback(self, data: dict):
        self.conn.execute("""
            INSERT INTO memory_log (account, risk_score, risk_level, recommendation_title, dec
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """, (
            data.get("account"), data.get("risk_score"), data.get("risk_level"),
            data.get("recommendation_title"), data.get("decision"),
            data.get("modification_notes"), data.get("outcome"), data.get("timestamp")
        ))
        self.conn.commit()

    def find_similar(self, risk_level: str, risk_score: float, top_n: int = 3) -> list[dict]:
        cursor = self.conn.execute("""
            SELECT * FROM memory_log WHERE risk_level = ? ORDER BY ABS(risk_score - ?) ASC LIM
        """, (risk_level, risk_score, top_n))
        cols = [d[0] for d in cursor.description]
        return [dict(zip(cols, row)) for row in cursor.fetchall()]

    def get_memory_context(self, risk_level: str, risk_score: float) -> MemoryContext:
        cases = self.find_similar(risk_level, risk_score, top_n=3)
        similar = len(cases)
        base = 0.73
        boosted = min(base * (1 + 0.06 * similar), 0.97)
        precedent = list({c["account"] for c in cases if c.get("account")})
```

```
        return MemoryContext(
            similar_cases_found=similar,
            confidence_boost=round(boosted - base, 3),
            base_confidence=base,
            boosted_confidence=round(boosted, 3),
            precedent_accounts=precedent
        )
```

8. **scripts/seed_memory.py**

```
#!/usr/bin/env python3
import json, glob
from backend.memory.episodic import EpisodicMemory
from datetime import datetime

def main():
    mem = EpisodicMemory()
    for path in glob.glob("backend/data/resolved_cases/*.json"):
        with open(path) as f:
            case = json.load(f)
        mem.log_feedback({
            "account": case["account_name"],
            "risk_score": 0.75,
            "risk_level": "high",
            "recommendation_title": case["action_taken"],
            "decision": "accept",
            "modification_notes": case["csm_notes"],
            "outcome": case["outcome"],
            "timestamp": datetime.utcnow().isoformat()
        })
    count = mem.conn.execute("SELECT count(*) FROM memory_log").fetchone()[0]
    print(f"Seeded {count} resolved cases into memory.")
    assert count >= 6

if __name__ == "__main__":
    main()
```

Day 3 — Demo Prep + Eval

9. Run seeding

```
python scripts/seed_memory.py
```

Verify: `SELECT count(*) FROM memory_log` returns >= 6.

10. `tests/eval_scenarios.py`

```
import pytest
from backend.agents.graph import run

SCENARIOS = [
    {
        "account_id": "acme_corp",
        "transcript": "PASTE_ACME_TRANSCRIPT_HERE",
        "expected_risk_level": "critical",
        "expected_primary_action_keyword": "EBR",
    },
    {
        "account_id": "globex_corp",
        "transcript": "PASTE_GLOBEX_TRANSCRIPT_HERE",
        "expected_risk_level": "low",
        "expected_primary_action_keyword": "enterprise",
    },
    {
        "account_id": "techcorp",
        "transcript": "PASTE_TECHCORP_TRANSCRIPT_HERE",
        "expected_risk_level": "medium",
        "expected_primary_action_keyword": "onboarding",
    },
]

@pytest.mark.parametrize("scenario", SCENARIOS)
def test_scenario(scenario):
    result = run(scenario["transcript"], scenario["account_id"])
```

```
    assert result.risk_level == scenario["expected_risk_level"]
    assert scenario["expected_primary_action_keyword"].lower() in result.primary_recommendatio
```

11. `docs/demo_script.md`

Write a short script for P3's demo video:

- Scenario 1: Load Acme Corp -> expect critical risk -> accept recommendation.
- Scenario 2: Load Globex Corp -> expect low risk -> show expansion play.
- Scenario 3: Load TechCorp -> expect memory boost -> show confidence jump.

Checklist Before Handoff

- ☐ `data/` contains exactly 10 + 15 + 5 + 6 files
- ☐ `python scripts/ingest.py` runs and outputs count >= 80
- ☐ `python scripts/seed_memory.py` runs and outputs count >= 6
- ☐ `vector_store.search("churn risk", "knowledge_base", 3)` returns results
- ☐ `episodic.get_memory_context("high", 0.8)` returns a `MemoryContext`
- ☐ `pytest tests/eval_scenarios.py` passes (or P1 fixes agent issues)
